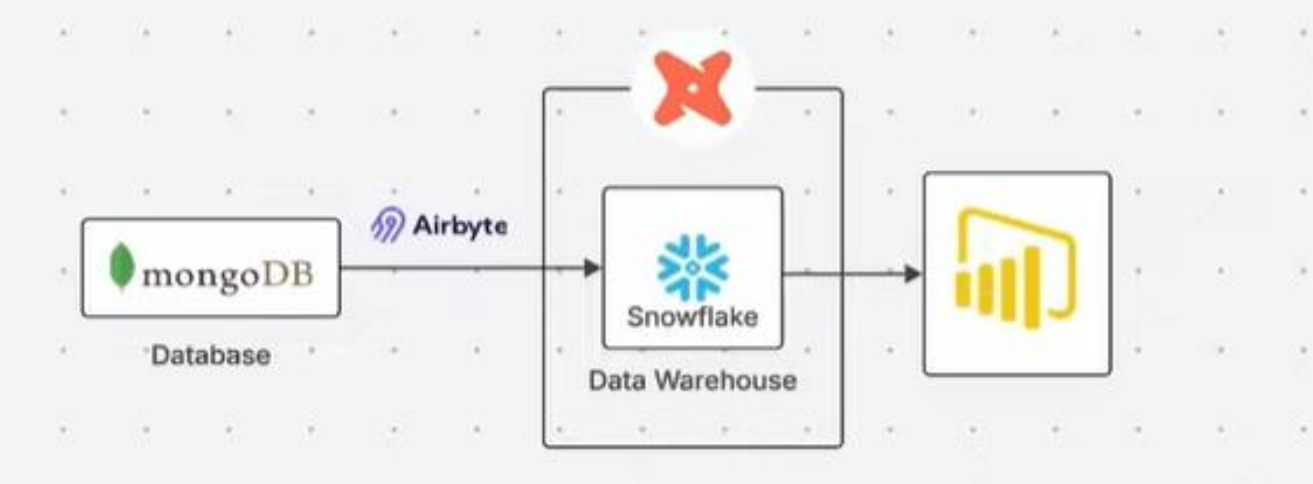
INSURANCE DATA ENGINEERING PIPELINE USING SNOWFLAKE AND DBT

- Aravind Swamy

Structure of the Pipeline:



Overview

This project implements a modern cloud-based data pipeline for insurance analytics, following the ELT (Extract, Load, Transform) pattern. The pipeline automates the flow of insurance data from a NoSQL operational database to a cloud data warehouse, where it is transformed and visualized for business insights.

Architecture Components

Data Generation and Source Layer

The pipeline begins with a Python script utilizing the Faker library to generate synthetic insurance data. This data is stored in MongoDB, which serves as the operational database (OLTP) for the insurance application. MongoDB's document-oriented structure handles semi-structured JSON data, simulating a real-world insurance management system.

Data Ingestion Layer - Airbyte

Airbyte acts as the data integration platform, establishing an automated connection between MongoDB and Snowflake. The tool is configured to sync data every three hours, ensuring the data warehouse remains updated with the latest operational data. Airbyte handles the extraction of JSON documents from MongoDB collections and loads them into Snowflake's raw layer without any transformations, following the ELT methodology.

Data Warehouse Layer - Snowflake

Snowflake serves as the cloud data warehouse with a two-schema architecture. The "raw" schema receives untransformed data directly from Airbyte, preserving the original data structure from MongoDB. The "analytics" schema contains transformed, business-ready data models created by dbt. This separation ensures data lineage and allows for reprocessing if needed.

Transformation Layer - dbt

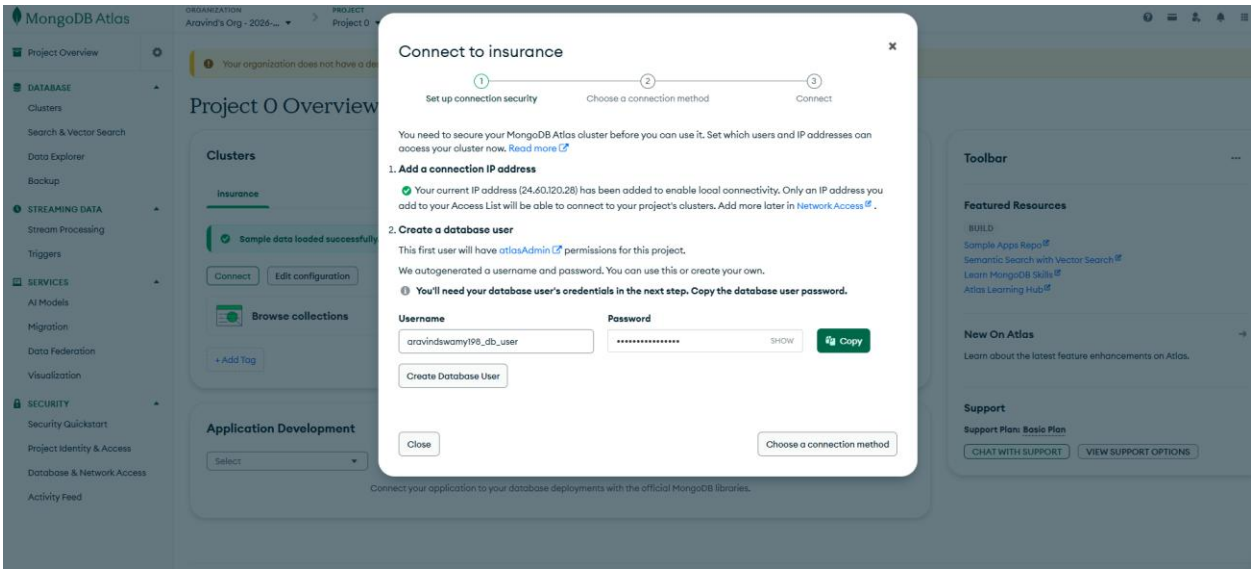
dbt (Data Build Tool) performs all data transformations within Snowflake using SQL. The transformations flatten the JSON structures from MongoDB, apply business logic, and create dimensional models optimized for reporting. All transformation logic is version-controlled and tested, with the final models materialized in the analytics schema for consumption by BI tools.

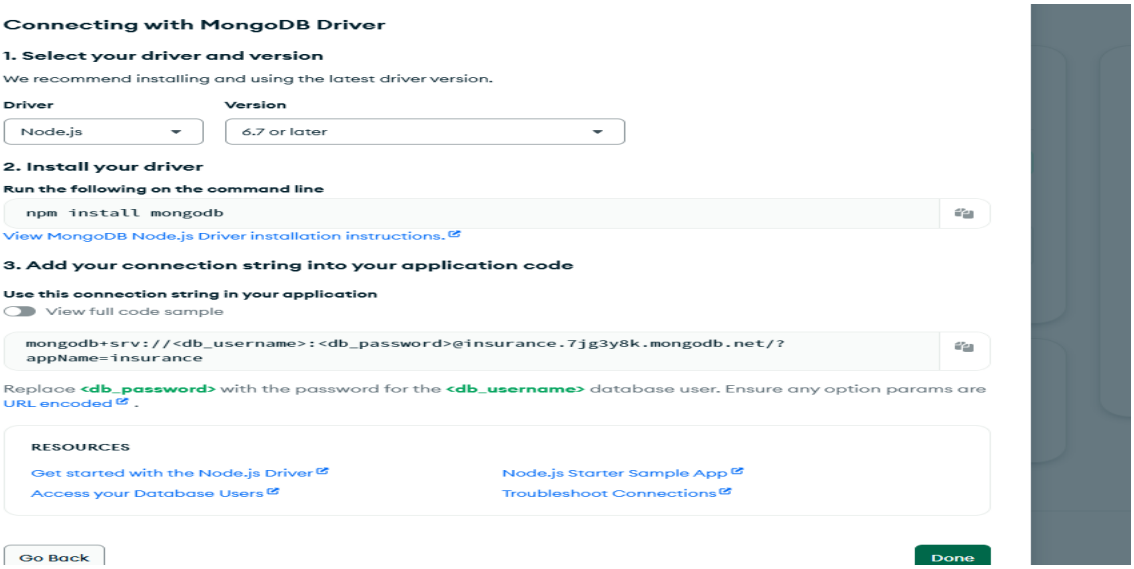
Visualization Layer - Power BI

Power BI connects to Snowflake's analytics schema to provide interactive dashboards and reports. The visualization layer enables business users to explore insurance metrics, trends, and insights without requiring technical knowledge of the underlying data structures.

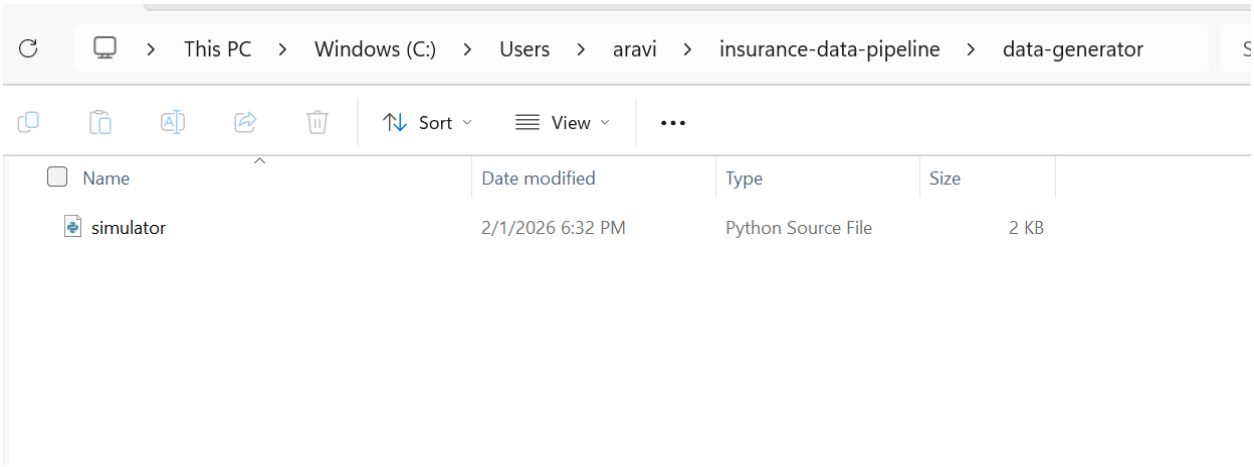
So now elaborating on each steps, we have

First, we create the MongoDB cluster for OLTP





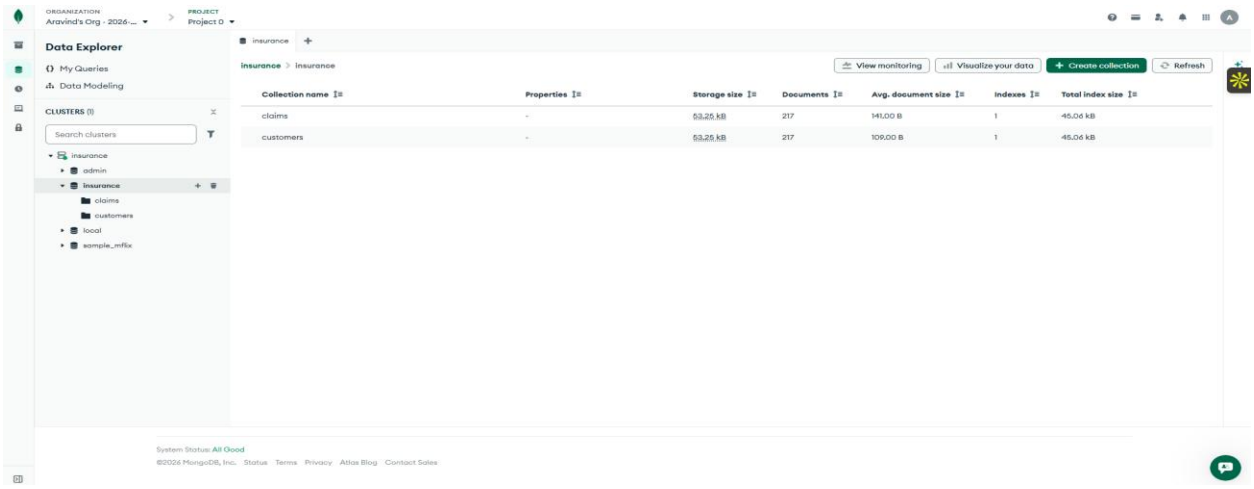
This is the simulator Python Faker code which will generate the mock data for our pipeline



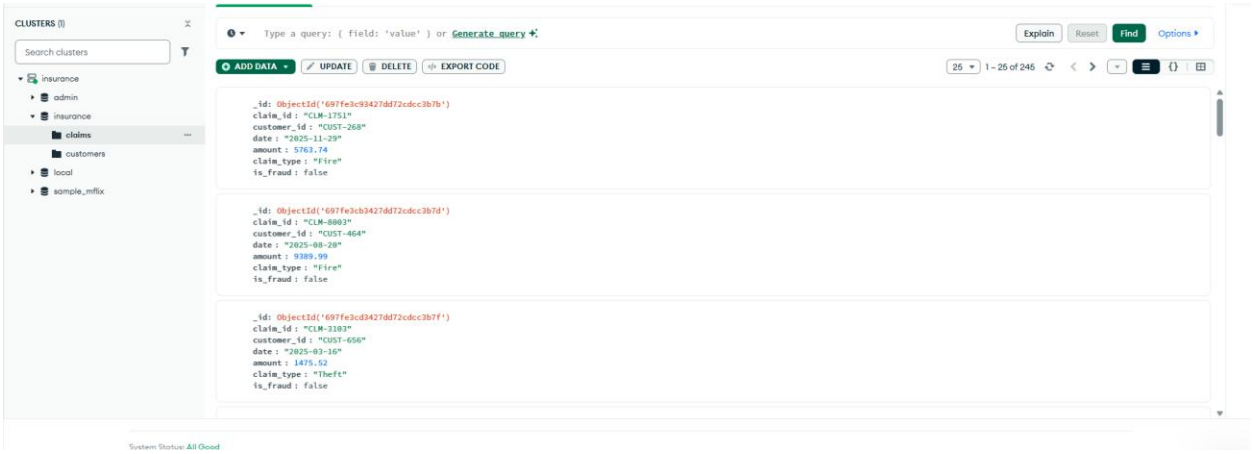
The data is thus generated,

```
C:\Users\aravi\insurance-data-pipeline\data-generator>python simulator.py
Inserted claim CLM-1751 for customer CUST-268
Inserted claim CLM-8003 for customer CUST-464
Inserted claim CLM-3103 for customer CUST-656
Inserted claim CLM-7026 for customer CUST-962
Inserted claim CLM-2552 for customer CUST-358
Inserted claim CLM-8306 for customer CUST-157
Inserted claim CLM-3025 for customer CUST-777
Inserted claim CLM-4181 for customer CUST-173
Inserted claim CLM-3438 for customer CUST-902
Inserted claim CLM-2424 for customer CUST-684
Inserted claim CLM-9440 for customer CUST-431
Inserted claim CLM-4801 for customer CUST-792
Inserted claim CLM-2610 for customer CUST-224
Inserted claim CLM-7425 for customer CUST-300
Inserted claim CLM-2179 for customer CUST-915
Inserted claim CLM-8527 for customer CUST-963
Inserted claim CLM-2255 for customer CUST-455
Inserted claim CLM-2565 for customer CUST-874
Inserted claim CLM-7798 for customer CUST-157
```

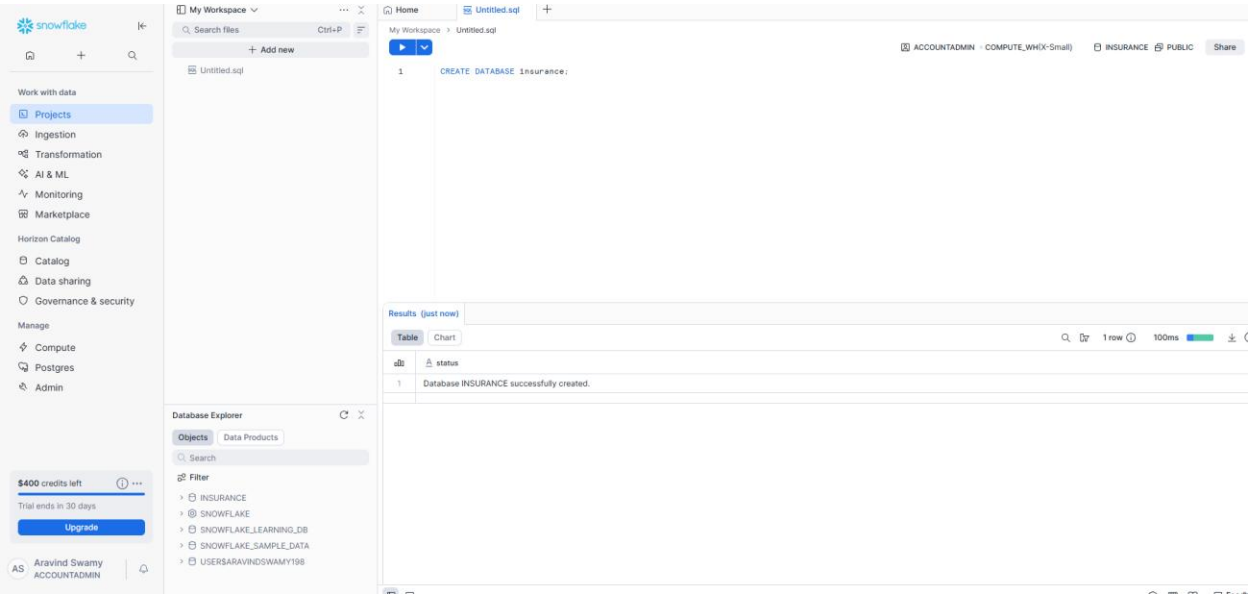
The data generated is loaded into our OLTP MongoDB database



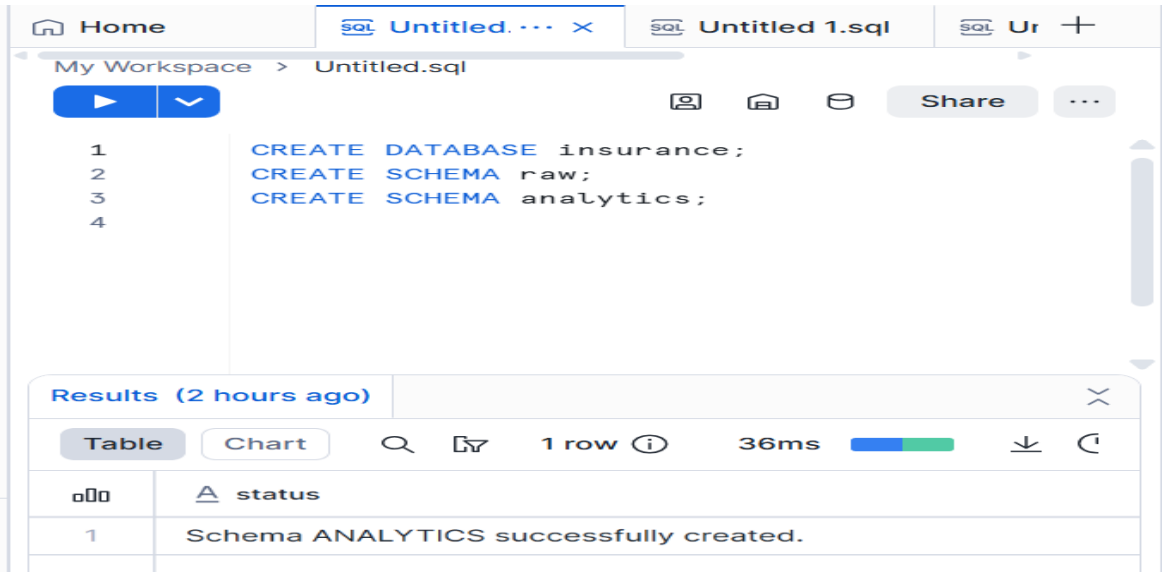
The structure of the data looks like this,



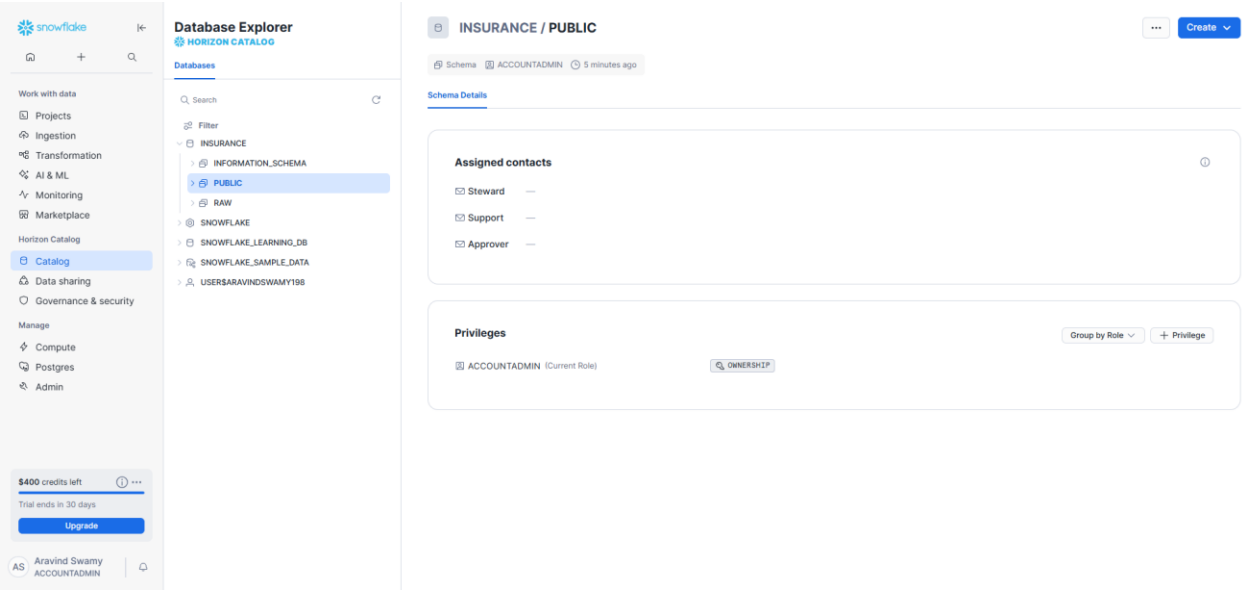
Now that the source database is successfully set up, we focus on the destination. We create a data warehouse database named insurance to store our data in the destination database before transformation



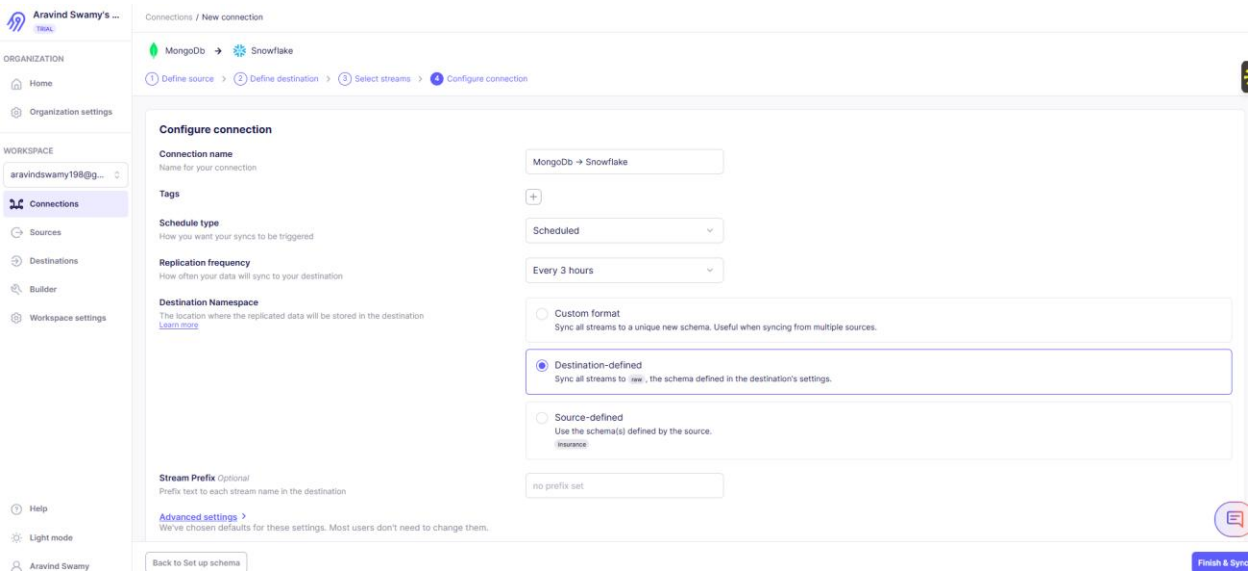
We also create two schemas. The raw schema is where we dump the data we get from the source, and the analytics schema is where we store the cleaned data, which we will use for analytics to gain insights.

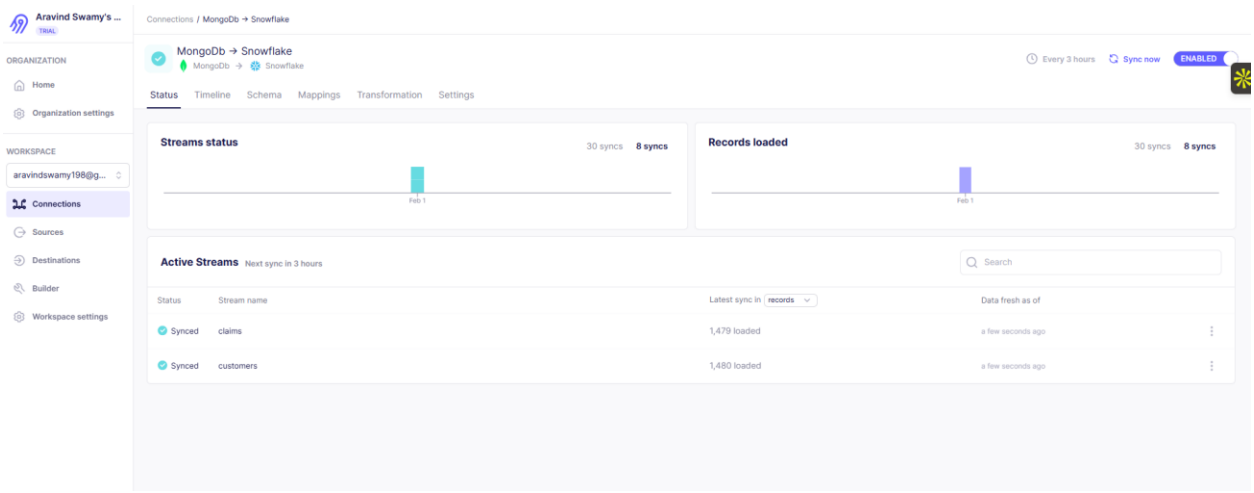


Now we have created the data warehouse DB and the schema raw where we will direct the data from the MongoDB to the snowflake datawarehouse insurance/raw



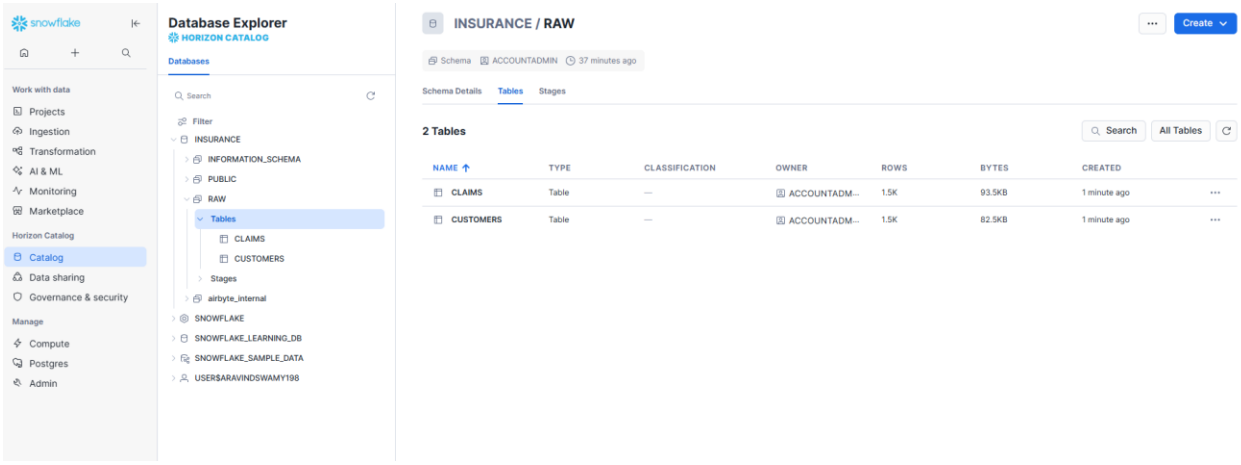
Now we use Airbyte to transfer the data from our MongoDB source database to our Snowflake data warehouse and we set the replication frequency as 3 hours so that every 3 hours the new data from the mongodb is pushed to our destination in snowflake.





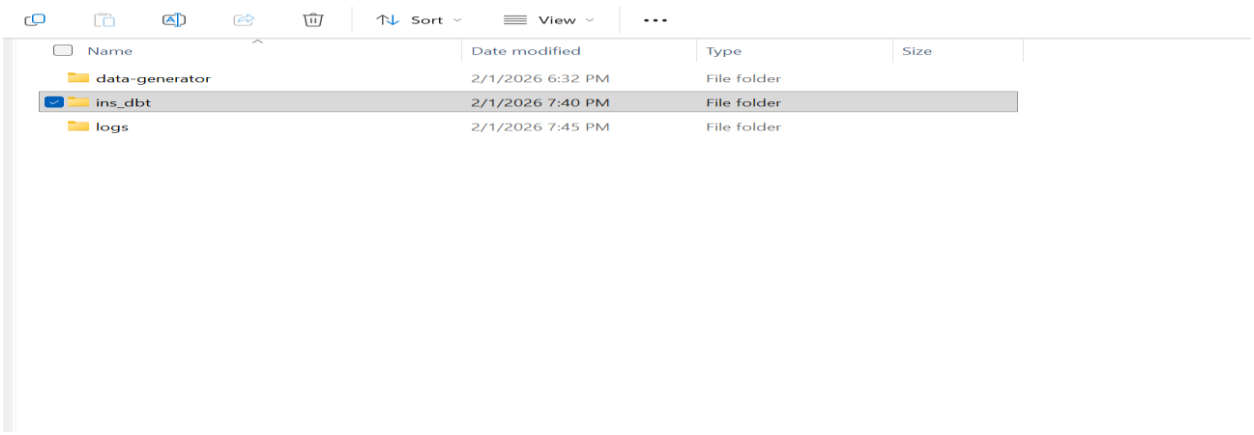
Now we can see that the data has been successfully transferred from our MongoDB to our data warehouse, Snowflake, and as mentioned, Airbyte here syncs our data warehouse with the database every three hours

Now we can see the data loaded in our raw schema

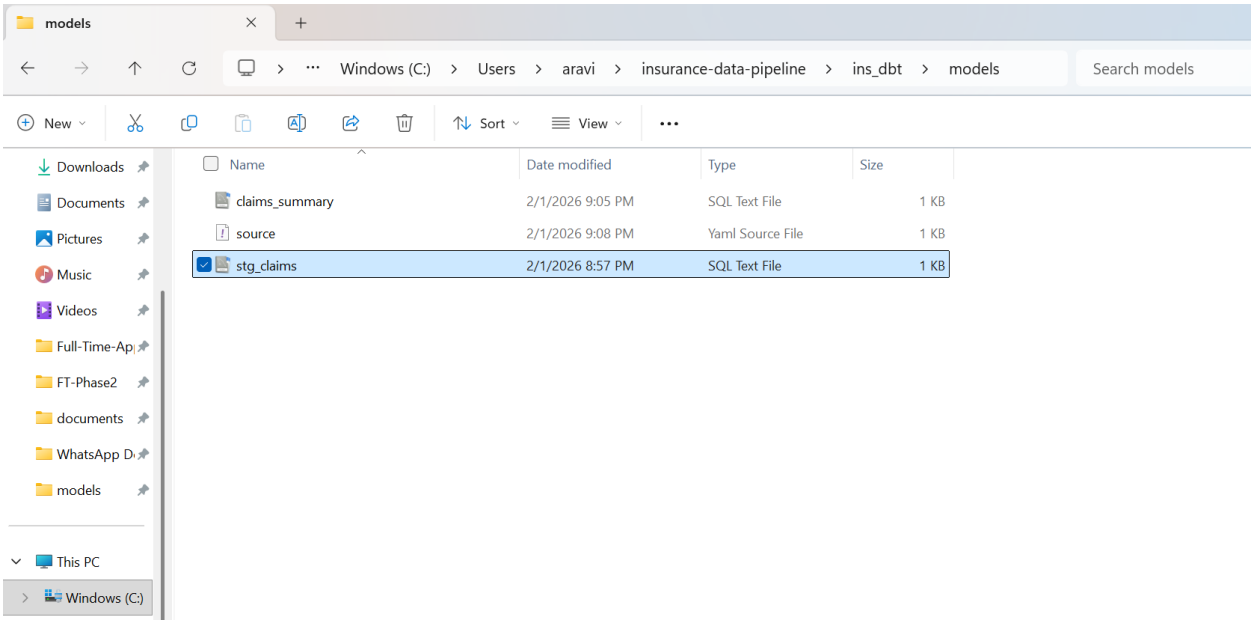


Now we go forward with dbt for our transformations (ELT)

```
C:\Users\aravi>pip install dbt-core dbt-snowflake
Collecting dbt-core
  Downloading dbt_core-1.11.2-py3-none-any.whl.metadata (4.4 kB)
Collecting dbt-snowflake
  Downloading dbt_snowflake-1.11.1-py3-none-any.whl.metadata (3.2 kB)
Collecting agate<1.10,>=1.7.0 (from dbt-core)
  Downloading agate-1.9.1-py2.py3-none-any.whl.metadata (3.2 kB)
Collecting click<9.0,>=8.2.0 (from dbt-core)
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting daff>=1.3.46 (from dbt-core)
  Downloading daff-1.4.2-py3-none-any.whl.metadata (10 kB)
Collecting dbt-adapters<2.0,>=1.15.5 (from dbt-core)
  Downloading dbt_adapters-1.22.5-py3-none-any.whl.metadata (4.5 kB)
Collecting dbt-common<2.0,>=1.37.2 (from dbt-core)
  Downloading dbt_common-1.37.2-py3-none-any.whl.metadata (4.9 kB)
Collecting dbt-extractor<=0.6,>=0.5.0 (from dbt-core)
  Downloading dbt_extractor-0.6.0-cp39-abi3-win_amd64.whl.metadata (4.6 kB)
Collecting dbt-protos<2.0,>=1.0.405 (from dbt-core)
  Downloading dbt_protos-1.0.427-py3-none-any.whl.metadata (859 bytes)
Collecting dbt-semantic-interfaces<0.10,>=0.9.0 (from dbt-core)
  Downloading dbt_semantic_interfaces-0.9.0-py3-none-any.whl.metadata (2.6 kB)
Requirement already satisfied: jinja2<4,>=3.1.3 in c:\users\aravi\appdata\local\programs\python\python312\lib\site-packages (from dbt
-core) (3.1.3)
Requirement already satisfied: jsonschema<5.0,>=4.19.1 in c:\users\aravi\appdata\local\programs\python\python312\lib\site-packages (f
rom dbt-core) (4.21.1)
Collecting mashumaro<3.15,>=3.9 (from mashumaro[msgpack]<3.15,>=3.9->dbt-core)
  Downloading mashumaro-3.14-py3-none-any.whl.metadata (114 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 114.4/114.4 kB 2.2 MB/s eta 0:00:00
Collecting networkx<4.0,>=2.3 (from dbt-core)
  Downloading networkx-3.6.1-py3-none-any.whl.metadata (6.8 kB)
```



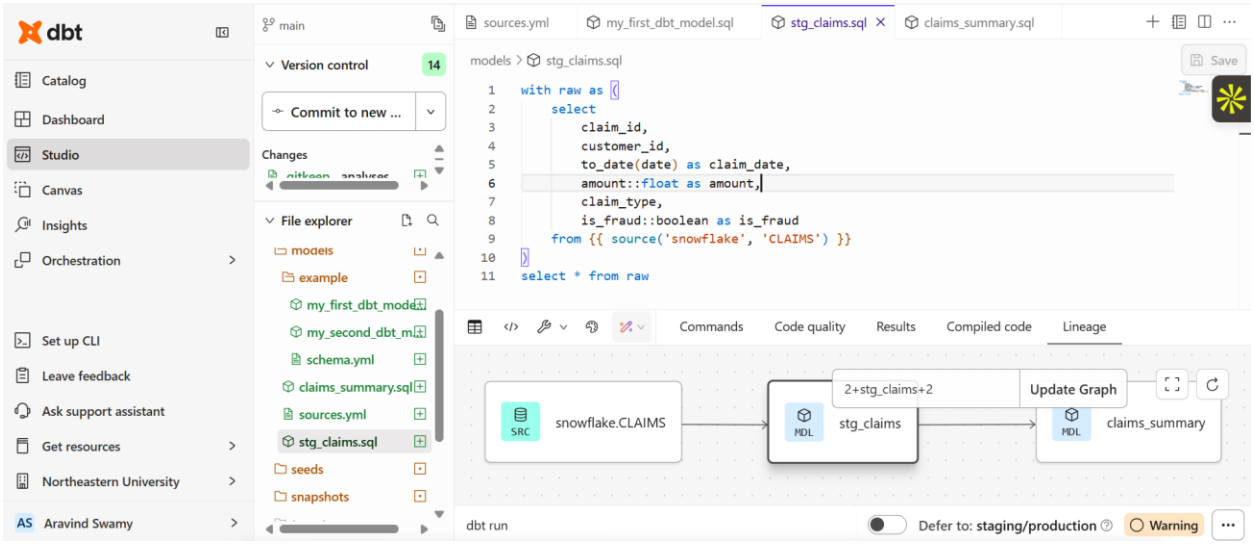
```
C:\Users\aravi\insurance-data-pipeline>dbt init ins_dbt
01:00:21  Running with dbt=1.11.2
01:00:21  A project called ins_dbt already exists here.

C:\Users\aravi\insurance-data-pipeline>rmdir /s ins_dbt
ins_dbt, Are you sure (Y/N)? y

C:\Users\aravi\insurance-data-pipeline>dbt init ins_dbt
01:25:36  Running with dbt=1.11.2
01:25:36
Your new dbt project "ins_dbt" was created!

For more information on how to configure the profiles.yml file,
please consult the dbt documentation here:

  https://docs.getdbt.com/docs/configure-your-profile

One more thing:

Need help? Don't hesitate to reach out to us via GitHub issues or on Slack:

  https://community.getdbt.com/

Happy modeling!

01:25:36  Setting up your profile.
Which database would you like to use?
[1] snowflake

(Don't see the one you want? https://docs.getdbt.com/docs/available-adapters)

Enter a number: 1
account (https://<this_value>.snowflakecomputing.com): rw54058.ca-central-1.aws
user (dev username): aravindswamy198
[1] password
[2] keypair
[3] sso
Desired authentication type option (enter a number): 1
password (dev password):
role (dev role): accountadmin
warehouse (warehouse name): compute_wh
database (default database that dbt will build objects in): insurance
schema (default schema that dbt will build objects in): analytics
threads (1 or more) [1]: 1
01:34:50  Profile ins_dbt written to C:\Users\aravi\.dbt\profiles.yml using target's profile_template.yml and your supplied values. Run 'dbt debug' to validate the connection.

C:\Users\aravi\insurance-data-pipeline>
```

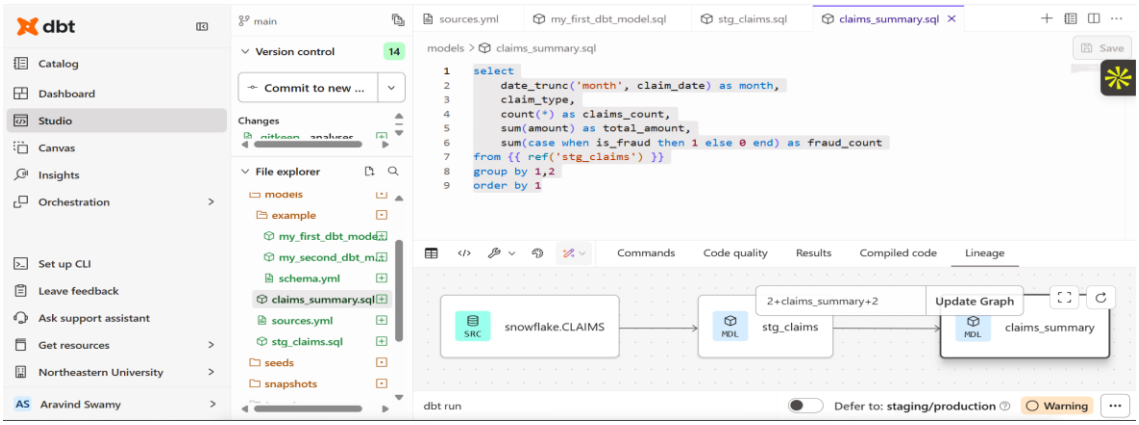


Now we run the dbt transformations. The transformation logic is stored in the files stg_claims and claims_summary

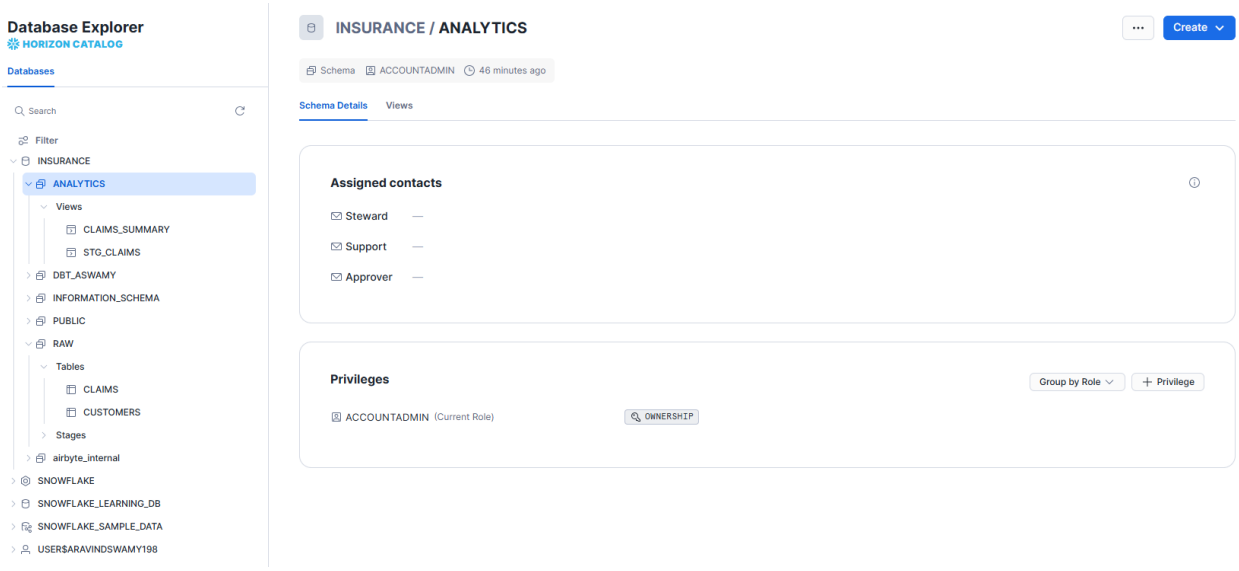
```
C:\Users\aravi\insurance-data-pipeline\ins_dbt>dbt run
02:13:15  Running with dbt=1.11.2
02:13:17  Registered adapter: snowflake=1.11.1
02:13:17  Unable to do partial parsing because saved manifest not found. Starting full parse.
02:13:19  [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
There are 1 unused configuration paths:
- models.ins_dbt.example
02:13:19  Found 2 models, 1 source, 522 macros
02:13:19
02:13:19  Concurrency: 1 threads (target='dev')
02:13:19
02:13:28  1 of 2 START sql view model analytics.stg_claims ............................... [RUN]
02:13:29  1 of 2 OK created sql view model analytics.stg_claims .......................... [SUCCESS 1 in 1.24s]
02:13:29  2 of 2 START sql view model analytics.claims_summary ........................... [RUN]
02:13:29  2 of 2 OK created sql view model analytics.claims_summary ...................... [SUCCESS 1 in 0.25s]
02:13:29
02:13:29  Finished running 2 view models in 0 hours 0 minutes and 10.45 seconds (10.45s).
02:13:30
02:13:30  Completed successfully
02:13:30
02:13:30  Done. PASS=2 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=2

C:\Users\aravi\insurance-data-pipeline\ins_dbt>
```

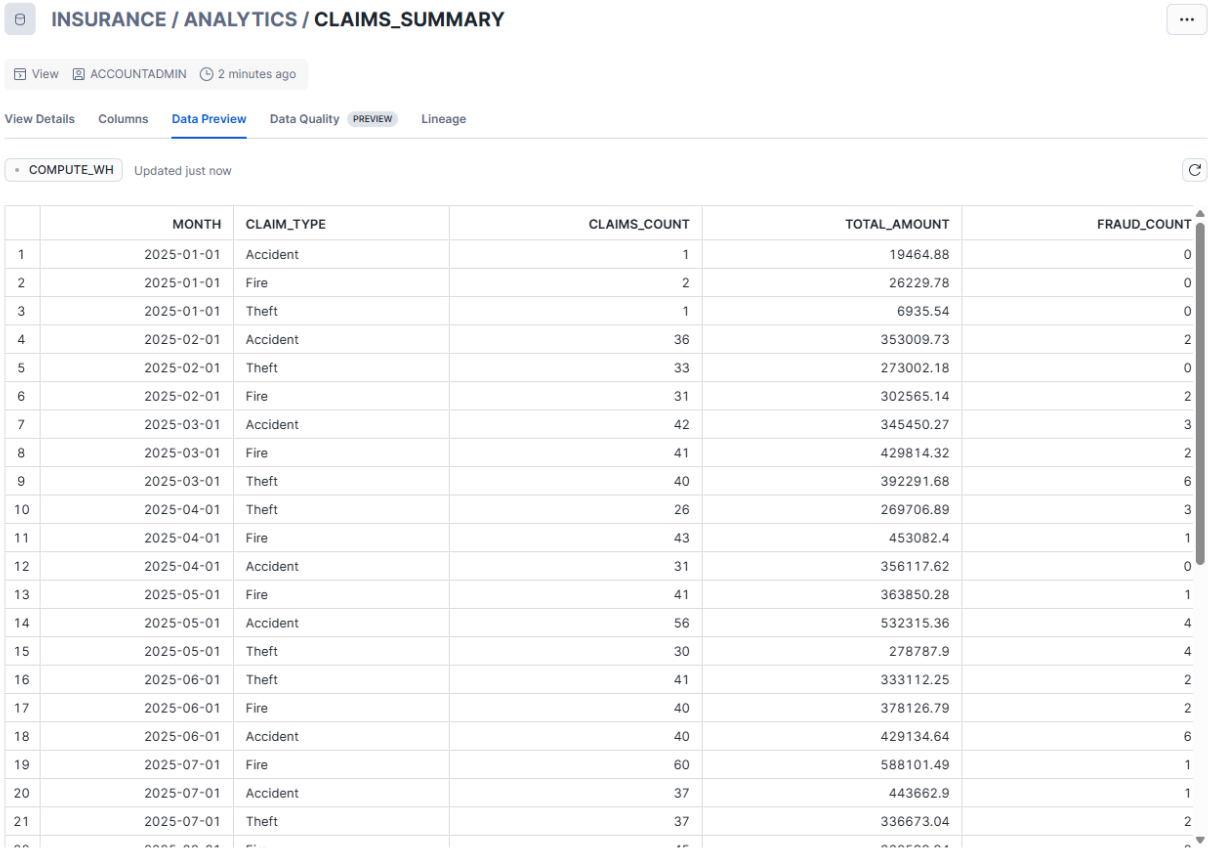
I also tried the same using dbt cloud in addition to running it from the CLI





Now we can see it under the analytics schema in our snowflake after the transformation,



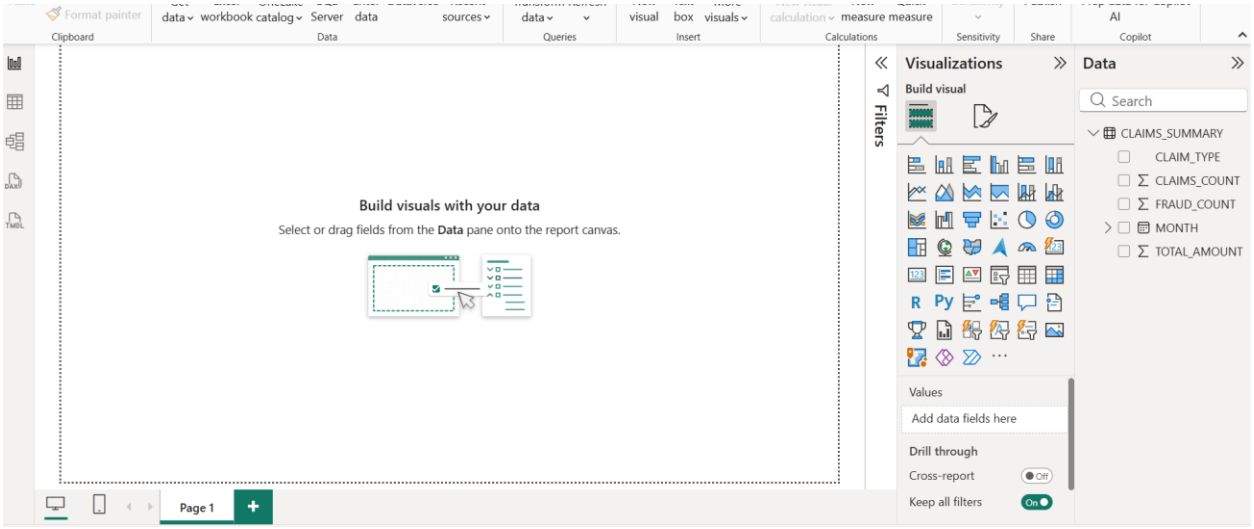


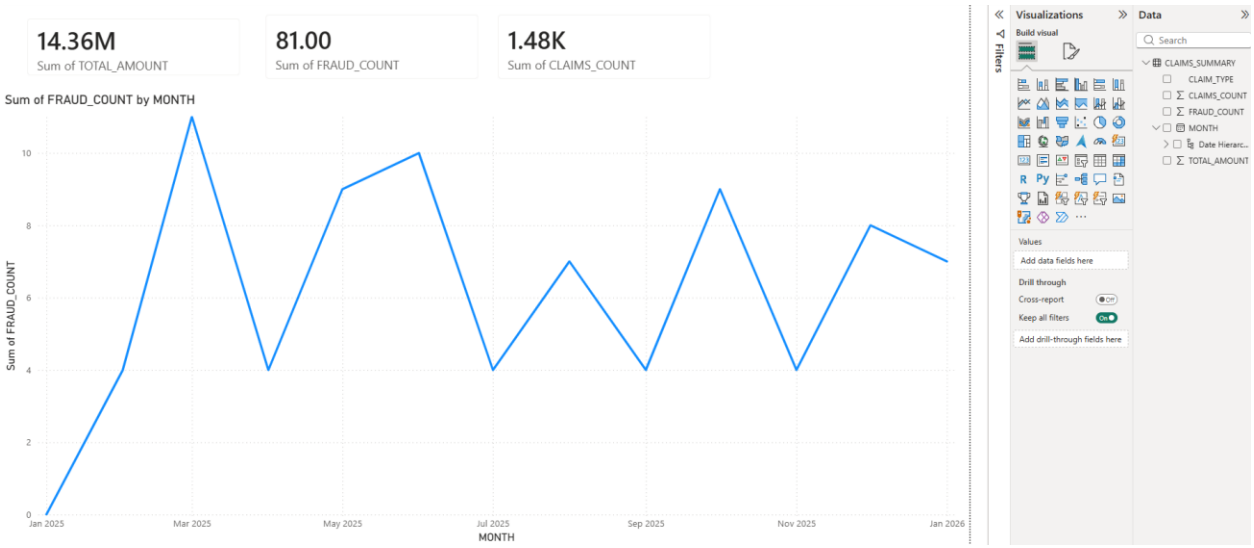
	MONTH	CLAIM_TYPE	CLAIMS_COUNT	TOTAL_AMOUNT	FRAUD_COUNT
1	2025-01-01	Accident	1	19464.88	0
2	2025-01-01	Fire	2	26229.78	0
3	2025-01-01	Theft	1	6935.54	0
4	2025-02-01	Accident	36	353009.73	2
5	2025-02-01	Theft	33	273002.18	0
6	2025-02-01	Fire	31	302565.14	2
7	2025-03-01	Accident	42	345450.27	3
8	2025-03-01	Fire	41	429814.32	2
9	2025-03-01	Theft	40	392291.68	6
10	2025-04-01	Theft	26	269706.89	3
11	2025-04-01	Fire	43	453082.4	1
12	2025-04-01	Accident	31	356117.62	0
13	2025-05-01	Fire	41	363850.28	1
14	2025-05-01	Accident	56	532315.36	4
15	2025-05-01	Theft	30	278787.9	4
16	2025-06-01	Theft	41	333112.25	2
17	2025-06-01	Fire	40	378126.79	2
18	2025-06-01	Accident	40	429134.64	6
19	2025-07-01	Fire	60	588101.49	1
20	2025-07-01	Accident	37	443662.9	1
21	2025-07-01	Theft	37	336673.04	2

Thus the the transformed data is written back to the analytics schema in snowflake.

Now we go on to visualize it in Power BI

Imported the data in power BI





Thus the entire pipeline is over

Conclusion

This project successfully demonstrates the implementation of a modern, cloud-native ELT pipeline for insurance data analytics using industry-standard tools and best practices. The pipeline efficiently automates the entire data lifecycle from generation through visualization, showcasing the capabilities of contemporary data engineering architectures.

The implementation leverages MongoDB as the operational database to handle semi-structured insurance data, with Airbyte providing reliable, automated data ingestion to Snowflake every three hours. The two-schema architecture in Snowflake ensures clear separation between raw and

transformed data, maintaining data lineage while enabling efficient reprocessing when needed. The use of dbt for transformations exemplifies the modern approach of performing data transformations within the warehouse, utilizing its computational power while maintaining version-controlled, testable transformation logic.

By implementing this pipeline using both dbt CLI and dbt Cloud, the project demonstrates versatility in working with different development environments and deployment strategies. The final integration with Power BI enables self-service analytics, allowing stakeholders to derive actionable insights from insurance data without requiring technical expertise.

This end-to-end solution highlights proficiency in Python programming, NoSQL databases, cloud data warehousing, modern ELT orchestration, analytics engineering, and business intelligence visualization. The architecture is scalable, maintainable, and follows industry best practices, making it suitable for production deployment in real-world insurance analytics scenarios.